

Lab 04 Report

Nathan Richins

A02285148

Anthony Wood

A02306431

Objectives

The purpose of this lab is to provide practice with meta-heuristic methods of optimization, specifically simulated annealing as it relates to placement of complex logic blocks (CLBs) on an FPGA.

Introduction

Timing requirements are an important part of any hardware design. To ensure that all operations perform as expected, the critical path must be short enough to allow the signal propagation to go from source register to destination register in under the period of the system clock. A major factor in ensuring the critical path meets the requirements is the physical distance from source to destination, which introduces the issue of placement.

FPGAs use CLBs to generate outputs. This allows for the generalization of hardware designs, and the ability to simulate physical circuit layouts. However, an FPGA has limitations on the arrangement of those CLBs. Thus we must come up with an arrangement of CLBs on the grid that attempts to minimize the path length. The first idea is to do an exhaustive search of all possible placement configurations. Define n to be the number of cells in the FPGA grid, and k to be the number of CLBs to place on the grid. We must choose without replacement, thus the number of possible orientations becomes $nPk = \frac{n!}{(n-k)!}$.

Number of Vertices	nPk	Duration
1	16	16 μ s
2	240	240 μ s
3	3360	3.36 ms
...	...	...
15	20922789888000	20922789.89 s
16	20922789888000	20922789.89 s

Table 1. The number of permutations to place k vertices on a 4x4 grid

Looking at Table 1, we can quickly see that for even a simple graph of 4x4, the number of orientations explodes into becoming impossible. Assuming we could check every permutation in 1 microsecond, with 16 vertices it would take around two-thirds of a year, and this is for a simple 4x4 grid. The DE10 Lite has around 5000 possible cells to place onto, so checking exhaustively is infeasible. Thus, we must begin using meta-heuristics to get a solution within a more feasible timeline. One such method is simulated annealing.

Simulated annealing is a method of optimization that imitates the annealing of metal. It starts at a high “temperature,” and allows for a large number of changes and moves in solutions. As the problem “cools,” we become more and more unlikely to take a suboptimal solution, until eventually we drop below our threshold temperature. These random changes allow us to avoid local optima, as we have higher energy shifts until we get towards the end of the annealing process. Algorithm 1 shows the actual process of simulated annealing.

```
Pseudocode for Simulated Annealing:
Begin
  generate initial solution
  score initial solution
  set initial temperature (T)
  Loop
    generate new solution
    score new solution
    If new better than old
      replace old solution with new
    Else
      compute  $\Delta E$  ( $\Delta E = |\text{scoreold} - \text{scorenew}|$ )
      compute acceptance probability ( $p = e^{-\Delta E/T}$ )
      generate random probability (r)
      If ( $r \leq p$ )
        replace old solution with new
      EndIf
    EndIf
    Lower T
  EndLoop when T is below threshold
End
```

Algorithm 1. The simulated annealing algorithm

For our problem, we needed to define a method of altering the solution, and score the solution. The latter question is simple. Based on the digraph G , which describes the connections of each CLB, we define the following scoring function

$$\sum_{i=0}^{|G|} (l_i)^2 \text{ where } l_i \text{ is length of edge } i \text{ the graph.} \quad (1)$$

We use the square of the lengths to more heavily weight higher edges. This will cause our algorithm to consider a single long path to be more punishing than a series of small but not optimal edges. The alteration method is a bit trickier. For this lab, we developed 4 different alteration methods which are described more in depth in the procedure section. However, they all share the same idea of moving a random vertex to another position in the grid. The idea is to introduce some randomness into the placement, because our testing showed that purely procedural alterations are prone to local optima.

Procedure

For this lab, we wanted to test different methods of alteration and determine which ones may be better than others. We came up with 4, which we named “Naive,” “Conway,” “Shift,” and “Centroid.”

Naive

The naive method was simply taking two vertices, and swapping them in place. We avoided self swaps, so we hoped this would help reduce the number of wasted iterations. This method works fairly well, but is unable to change the structure, so there are some graph configurations which will never be significantly improved by using this naive alteration method.

Conway

This method is inspired by Conway’s Game of Life. The idea is that the graph has exterior positions which we might consider “padded,” These padded positions mark locations adjacent to some vertex on the grid, but themselves aren’t a part of the grid. This increases our range of possible destination values, but also lets us change the structure of the placement configuration.

Shift

The shift method still picks a random source vertex, but tries to minimize that vertex's largest edge. We identify the undirected neighbor to that vertex that has the largest edge, and try to place the vertex directly adjacent to it. If that position is filled, we either replace the vertex

that is there, or we shift out again. This occurs up to some maximum number of shifts, at which the vertex is guaranteed to be placed.

Centroid

The centroid method takes all neighbors into account, and attempts to place the vertex at the centroid of the polygon formed by the neighbors. This tries to reduce all edge lengths, and ideally would work to group vertices together. There is no fallback, so we place the vertex in an unoccupied cell, or replace that position based on some probability. If we fail that probability, there is no shift.

After determining these methods. We ensured that we could read in the file to actually place the vertices on the grid. This proved to be a simple task, but we did end up using the `<filesystem>` library introduced in C++17 to read it in. While it does introduce some version dependency, it makes reading the file reliable and straightforward.

With the graph read in, we just needed to execute the algorithm, and the code shown in Appendix A shows how simple this ended up being. The `mutationMethod` array and `restoreMethod` array were arrays of functions with the same function signature, allowing us to test different methods of alteration. Afterwards, we wrote the results to a file and exited. There are some visualization methods introduced, but were beyond the scope of the assignment and were used more for visual appeal than actual operation. Because of this, they are not discussed in this report.

Analysis

The cooling rate of the annealing process has a large influence on the operation of the simulation process. Higher cooling rates mean we run much faster, but are less likely to reach a solution. Conversely, lower cooling rates allow for better solutions, but can take much longer to converge. Figure 1 shows the final score versus runtime duration versus the cooling rate using the “Shift” method. The rates used were 0.9, 0.95, 0.99, 0.995, and so on up to 0.99999. We also did analysis on the 3 other methods, but those figures are not discussed. Their form is similar, so we will just discuss the one. The others can be found at the repo and in Appendix A.

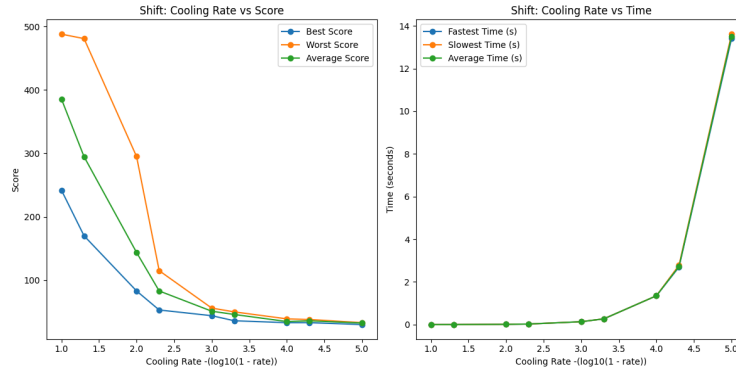


Figure 1. The cooling rate analysis plots against score and runtime duration

To introduce a more even independent scale, we decided to use a transformation of $-\log_{10}(1.0 - \text{rate})$. This allows the spacing to be even, but the rate to continue to rise from smallest rate to highest. It also helps the charts be more readable and interpretable. We can see that the lower few rates (0.9, 0.95, and 0.99) aren't able to produce good results. The state cools too quickly, and we can't get better solutions in time. However, as we drop down to 0.995 and 0.999, we can see that the score (squared distance) drops significantly. Inversely, the duration begins to increase as we hit these rates, and will exponentially begin to do so. This stands to reason, as our cooling is geometric, and so should end up with a form of exponential increase of duration. We can also see that right around 0.999 and 0.9995, we have a fairly good score, while still staying under 2 seconds. In fact, nothing below 0.99999 takes particularly long, but still converges well. So it's interesting to see how well we were able to converge within such a short time span.

It's likely the functions internally could be optimized to allow for more efficient runtimes. For instance, the score was generated quite naively by just iterating over every edge, and then squaring and summing that difference. However, since after an alteration occurs, the only affected edges are the ones associated with the two edges, a lot of effort is wasted. We could have just changed the corresponding edge changes, and then adjusted the score in place rather than continuously recomputing. In general though, the process appears to run well, and as expected, there is a tradeoff between accuracy and speed.

Results

We were able to generate a way of simulating the placement problem using simulated annealing. On average, we noticed that the "Naive" method would do ok, but the "Conway" and "Shift" methods would perform quite well, and approach the most optimal solution. The "Centroid" method was too prone to forcing itself into a structure that struggled to optimize to be effective. Figure 2 shows the score of the "Shift" method on the first input file `input1.txt` provided in the assignment description on Canvas. Figure 3 shows the best all time score for those runs

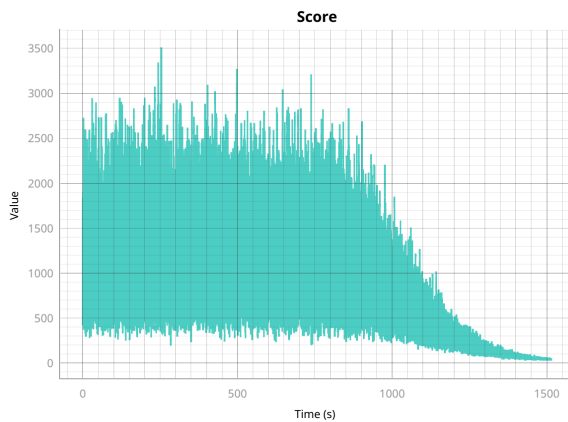


Figure 2. Score versus time using “Shift” on input1.txt

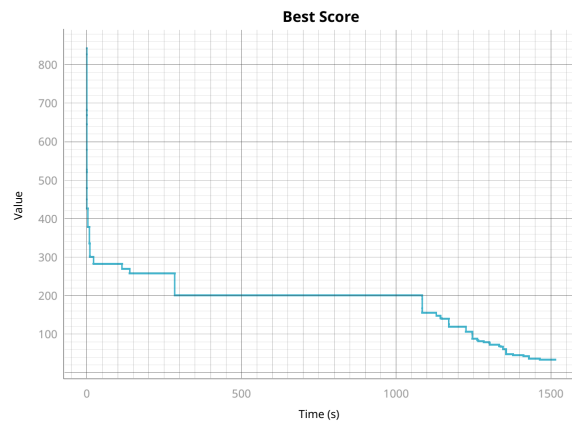


Figure 3. Best score using “Shift” on input1.txt

We can see the volatility in the score in the initial 70% of the run. Then as we reach approximately 900-1000 milliseconds, the volatility decreases as our “temperature” decreases. This makes sense, as we’re less likely to accept a worse solution over time. Interestingly though, we also see our floor drop as well. I think this comes from not allowing ourselves to swap out of a good configuration, so we get into a good spot, and then make smaller adjustments that improve the overall structure. This hypothesis is supported by the best scores plateauing around 300ms up to 1100ms. We’re too willing to accept poorer solutions, and so place ourselves in situations where we can’t improve the structure. The other methods have a similar shape, though drop at different rates and times. I’m not certain what drives when the plot will drop or by how much, just that it is consistently happening.

There is too much source code to place with an appendix reasonably. Thus, a link to the github repo containing the source code can be found [here](#). The code is also included in the zip file, but for convenience while looking at the report, the repo code is available.

Conclusion

Optimization is an important part of any engineering project. However, as problems scale, optimizing them becomes increasingly difficult. By using meta-heuristic methods such as simulated annealing, we can come to good solutions in a much more reasonable amount of time. Additionally, we have the ability to quit when we reach a good enough solution, rather than having to force a pipeline to optimality. This lab provided a good way of practicing implementing simulated annealing, but also gave us a greater understanding of what goes on in tools like Quartus when we synthesize hardware designs. Overall, this lab was challenging, interesting, and provided valuable context for working with FPGAs.

Appendix A: Cooling Analysis Figures

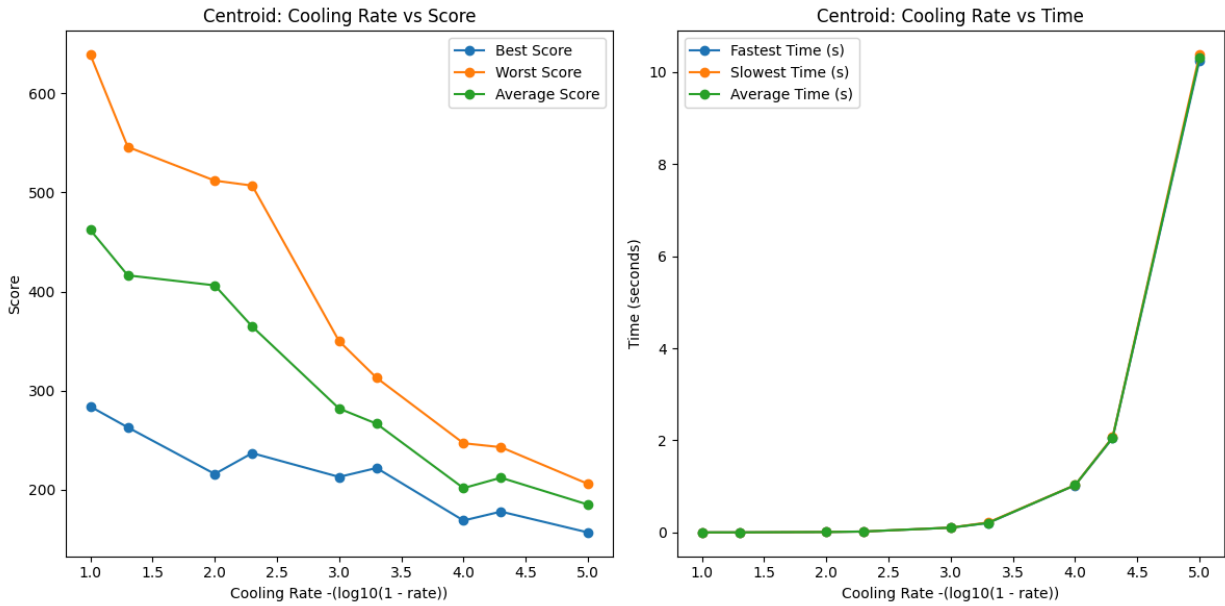


Figure 4. Centroid Cooling VS Score and Time

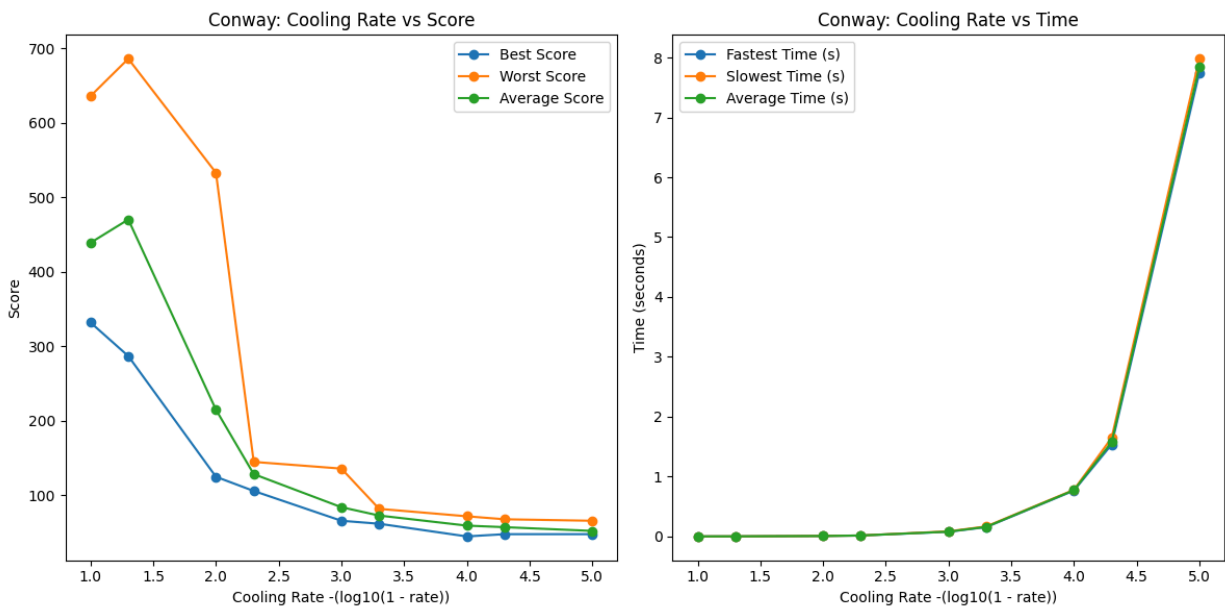


Figure 5. Conway Cooling VS Score and Time

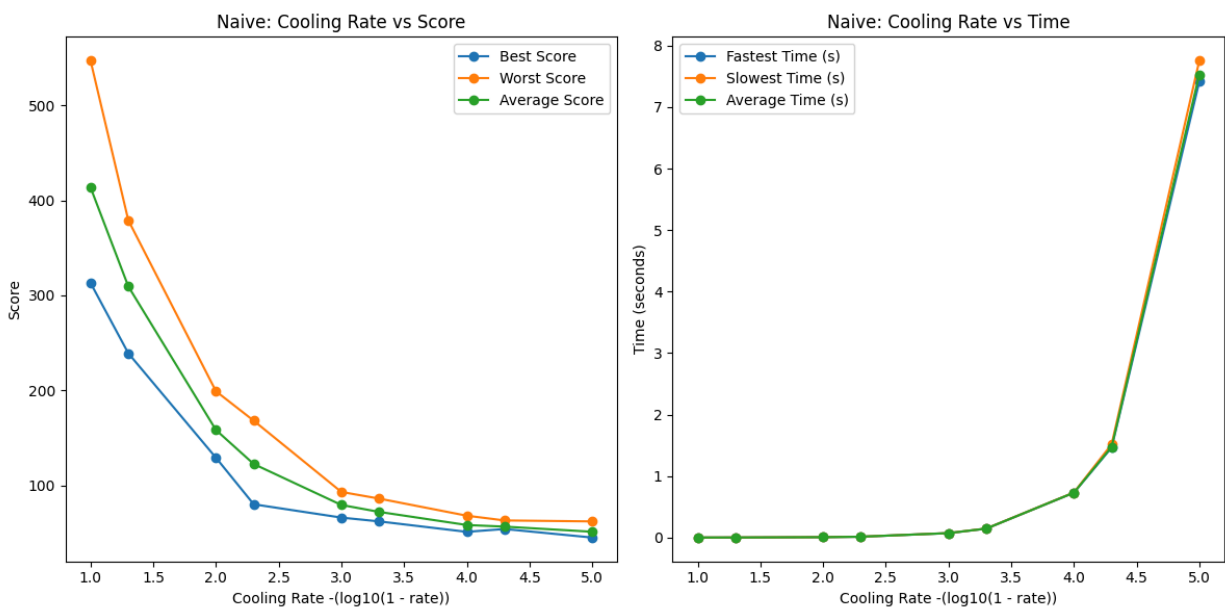


Figure 6. Naive Cooling VS Score and Time

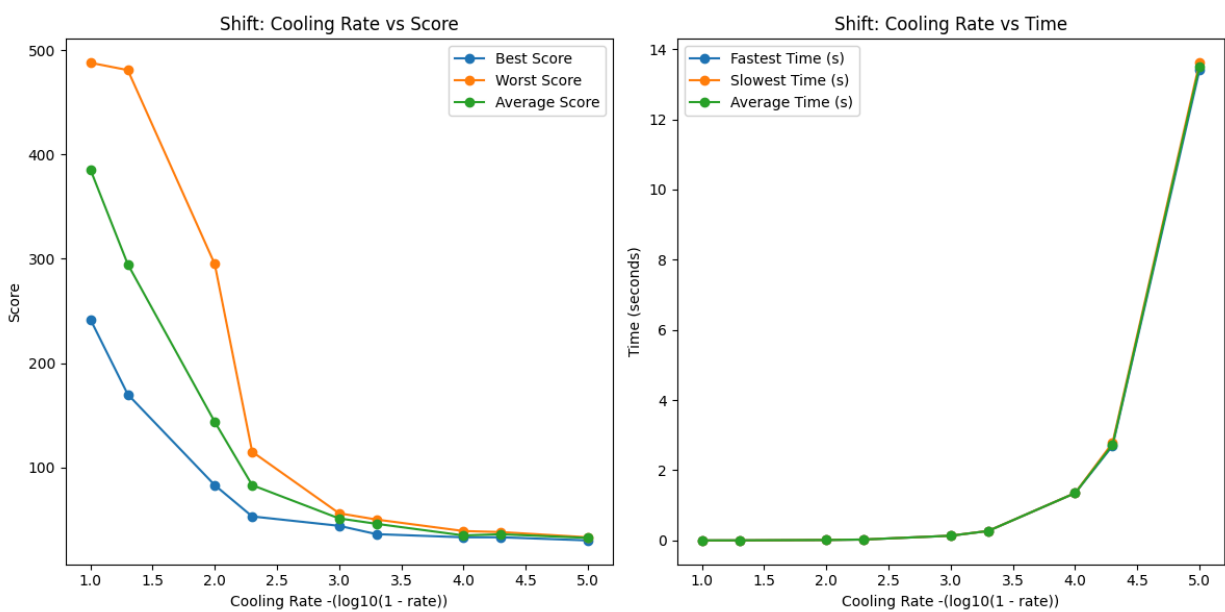


Figure 7. Shift Cooling VS Score and Time